

A Ratio-Closure Simplex for Maximum-Ratio Closure and its Canonical Decomposition

A didactic derivation on general directed acyclic graphs

Working document for mathematical review

5 September 2026

Abstract

This tutorial develops the ratio-closure simplex for the maximum-ratio closure subproblem underlying a linear program with arbitrary precedence constraints, positive weights, profits of either sign, unit upper bounds, and one capacity constraint. The central object is a spanning forest of *nonbasic slack variables*, together with roots for its zero components. We derive the basis representation, oriented subtree formulas for reduced costs and directions, the ratio test, all four structural pivot updates, and an exact-arithmetic termination argument. This representation supports specialized computation of both entering-variable prices and the pivot column: all reduced costs take $O(n)$ arithmetic operations after the forest is available, and a full pivot takes $O(n + m)$ with a complete arc scan. Repeated ratio optimization recovers the block decomposition and an optimum of the capacitated LP. A complete eight-pivot trace includes six degenerate pivots and is checked independently with rational linear algebra. This is a mathematical and didactic draft, not a performance report or a claim that the optimized C++ solver has already been implemented.

The three structural ingredients. A basis is encoded by a forest and its roots, with exactly one rootless tree. This combinatorial object gives both the reduced costs used to choose an entering variable and the corresponding pivot column through subtree sums and signed component movements. These are specialized solutions of the two basis systems of the revised simplex method. Sections 3 and 4 establish these three ingredients; Section 8 summarizes the arithmetic cost of each operation. The input precedence graph may be any DAG: the forest represents the current basis.

Status and reading guide. The notation and general LP follow the supplied manuscript Dose et al. (2026). The forest viewpoint starts from the internal notes [Project archive](#) (n.d.); here its formulas are derived for arbitrary arc orientations. Sections 1–2 introduce only the background needed by the algorithm. Sections 3–6 are the core: basis, pricing, directions, pivots, and a worked execution. Section 7 returns to the original LP. The proofs concern exact arithmetic. The numerical and performance variants in Section 8 are implementation proposals, not measured results. The checks in Section 9 support review of the formulas; they do not replace the proofs or constitute a production solver. No novelty claim is made before comparison with the relevant simplex and closure literature.

Contents

1	The linear program and the running instance	4
1.1	Names and notation, taken from the literature	4
1.2	The problem: maximum-ratio closure	4

1.3	Application: adding a capacity row	5
1.4	Eight vertices, several precedence levels	5
1.5	The dual and a convention to remember	6
1.6	Where this sits in the literature	6
1.7	Cycles and limits of the model	7
2	The ratio oracle and its normalization	7
2.1	Normalization removes the quotient	8
2.2	The class of linear program this is	8
2.3	Standard form and the ratio dual	9
3	A basis represented by a forest	9
3.1	Nonbasic slacks are the forest edges	9
3.2	A simple feasible initialization	11
3.3	Rooting and the orientation sign	11
4	Reduced costs and directions from subtree sums	11
4.1	Connection with revised simplex: the two basis systems	11
4.2	The dictionary behind the formulas	12
4.3	A specialized pivot column from one component movement	12
4.4	Reading a dual certificate from the forest	14
5	The ratio test, forest updates, and termination	14
5.1	The largest admissible step	14
5.2	All four structural cases	14
5.3	A finite reference algorithm	15
6	A complete execution on the running instance	16
6.1	Variable order and the eight pivots	16
6.2	A first zero step: a prerequisite blocks motion	16
6.3	The principal split: pivot 7	17
6.4	An optimal point need not yet have an optimal basis	17
7	From the forest oracle to the capacitated LP	18
7.1	Residual extraction and decreasing ratios	18
7.2	Why a single basis may miss a canonical block	18
7.3	The canonical LP solution	19
7.4	Finishing the eight-node example	20
8	Efficient implementation: what the derivation permits	20
8.1	A linear-memory reference organization	21
8.2	Specialized operations in one iteration	21
8.3	Variants to implement and tune	22
8.4	Degeneracy is the dominant cost	22
8.5	The dual side	23
8.6	Numerical checks and output contracts	24
9	What has been checked, and how to review this draft	25
9.1	Independent exact checks	25
9.2	Rebuilding the tables and PDF	25
9.3	Suggested order for mathematical review	26
9.4	Work remaining after this review version	26

A Closure and min-cut as comparison tools**26**

1 The linear program and the running instance

Let $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ be a directed acyclic graph (DAG), with $n = |\mathcal{V}|$ vertices and $m = |\mathcal{A}|$ arcs. An arc (i, j) means that *vertex j is a prerequisite of vertex i* . Its inequality is therefore $x_i \leq x_j$. This convention is used in every formula and diagram below. Each vertex has profit $p_i \in \mathbb{R}$ and weight $w_i > 0$. There is a single vertex type: no bipartition or alternation of vertex types is assumed, a vertex may have prerequisites and itself be a prerequisite, and profits may be negative, so selecting a profitable vertex can require selecting loss-making predecessors.

Definition 1.1 (Closure). A set $C \subseteq \mathcal{V}$ is a closure if $i \in C$ and $(i, j) \in \mathcal{A}$ imply $j \in C$. Write $\mathcal{C}$ for all closures, including the empty set. For $S \subseteq \mathcal{V}$, let

$$p(S) = \sum_{i \in S} p_i, \quad w(S) = \sum_{i \in S} w_i, \quad \varrho(S) = \frac{p(S)}{w(S)} \quad (S \neq \emptyset).$$

1.1 Names and notation, taken from the literature

Every term used below already has an owner, and this tutorial uses the owner's word rather than a new one.

Object	Name used here	Source
$\mathcal{G} = (\mathcal{V}, \mathcal{A})$	digraph, vertices, arcs	standard
$C \subseteq \mathcal{V}$ down-closed	<i>closure</i>	Picard (1976)
$\max_C p(C)/w(C)$	<i>maximum-ratio closure</i>	Hochbaum (2024)
the same set, order-theoretic	ρ -maximal <i>initial set</i>	Sidney (1975); Lawler (1978)
the nested chain of closures	<i>Sidney decomposition</i>	Sidney (1975)
its successive differences $\mathcal{K}_r$	<i>blocks</i>	this tutorial
forest with one distinguished vertex per tree	<i>rooted forest, roots</i>	Yang (2026)
the tree with no such vertex	<i>rootless tree</i>	Yang (2026)
forest with per-branch aggregates	<i>normalized tree</i>	Lerchs and Grossmann (1965)

Picard (1976) writes *maximal* closure where current usage writes *maximum* closure; the latter is used here. Matrices and vectors are set in bold, scalars and indexed entries are not, so $\mathbf{w}$ is the weight vector and w_i one of its entries.

1.2 The problem: maximum-ratio closure

The ratio-closure simplex solves one problem, and it is a *ratio* problem over closures:

$$\rho^* = \max \left\{ \varrho(C) = \frac{p(C)}{w(C)} : \emptyset \neq C \subseteq \mathcal{V}, C \in \mathcal{C} \right\}. \quad (1.1)$$

Note what is absent: there is no capacity, and no upper bound on any variable. The name of the method follows (1.1), in the way that *network simplex* names the class of flow problems rather than the spanning tree it maintains.

Two devices linearize (1.1). Normalizing the denominator (Charnes and Cooper, 1962) gives a single linear program, stated as (2.2) and proved equivalent in Theorem 2.1; that program, and its structure, is what the rest of this tutorial works on, and Section 2.2 identifies the class it belongs to. Treating the ratio parametrically instead (Dinkelbach, 1967) turns each iterate into a maximum closure and hence a minimum cut (Picard, 1976); that is the route of the comparison methods in Section A, not of the method developed here.

1.3 Application: adding a capacity row

The program (1.1) is an oracle, and the problem it serves is the LP relaxation of a precedence-constrained knapsack. For capacity $c \geq 0$,

$$z(c) = \max \sum_{i \in \mathcal{V}} p_i x_i, \quad (1.2a)$$

$$\text{s.t.} \quad \sum_{i \in \mathcal{V}} w_i x_i \leq c, \quad (1.2b)$$

$$x_i - x_j \leq 0 \quad (i, j) \in \mathcal{A}, \quad (1.2c)$$

$$0 \leq x_i \leq 1 \quad i \in \mathcal{V}. \quad (1.2d)$$

The indicator χ^C is feasible for the precedence and box constraints exactly when C is a closure; the capacity row makes the LP generally fractional. Solving (1.2) for a given c needs no new machinery beyond (1.1): repeated calls of the oracle on residual graphs produce closures of decreasing ratio, and c decides how far along that sequence one walks and which fraction of the critical block is taken. Section 7 does this, and Theorem 7.4 gives the whole function $z(\cdot)$ at once, piecewise linear with one breakpoint per block, at no extra cost. The capacity therefore never appears inside the simplex; it appears only in the procedure that calls it, which is why the method is named after (1.1) and not after the knapsack.

The same machinery answers a question in which no capacity appears at all. Driving the oracle to exhaustion — extract a block, delete it, ask again on the residual until nothing is left — produces the *canonical decomposition*: the complete sequence $\mathcal{K}_1, \dots, \mathcal{K}_k$ of blocks of strictly decreasing ratio, together with the threshold of every vertex and the breakpoints of the parametric closure value. This is the object the method is built to deliver, and it is what the solver computes under the tasks **full-path** and **positive-path**; a single capacity is then just a query against a sequence already in hand. The sequence is not a device of this tutorial: it is the decomposition of [Sidney \(1975\)](#), and the same object is studied on precedence forests, under the name *closure layers*, by the companion work of this project. What is specific here is that a simplex basis, rather than a flow, is what walks it.

1.4 Eight vertices, several precedence levels

We use the running instance from the supplied manuscript. Its data are stored once in `tests/fixtures/paper_example8.json`; the tables and figures below are generated from that file. Its arcs, in the fixed order used to number the slack variables, are

i	1	2	3	4	5	6	7	8
p_i	1	-1	-1	-1	6	5	2	3
w_i	1	1	1	2	2	1	1	1

Table 1: Profits and weights of the running instance.

$$\mathcal{A} = \{(5, 1), (5, 2), (5, 6), (6, 3), (7, 3), (7, 4), (8, 5), (8, 6), (8, 7)\}. \quad (1.3)$$

For example, vertex 8 requires vertices 5, 6 and 7, which in turn impose other prerequisites. At $c = 4$, the closure $\{1, 3, 6\}$ is an optimal integer selection of value 5. The LP value will be 7. The integer problem is used only to illustrate the difference; it is not the problem solved here.

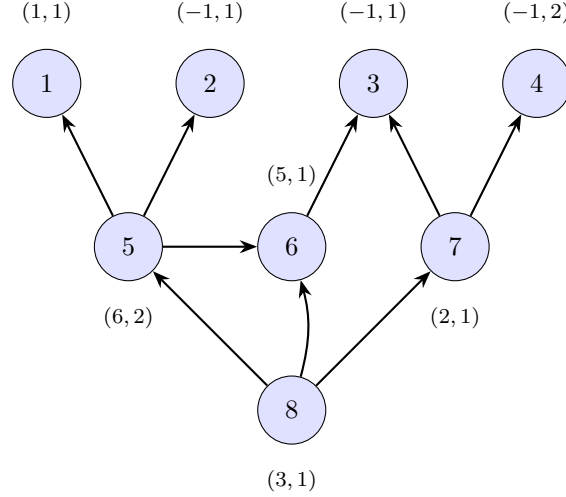


Figure 1: The general precedence graph. A label (p_i, w_i) is attached to each vertex; arrows point to prerequisites.

1.5 The dual and a convention to remember

For arc multipliers α , use the divergence convention of [Dose et al. \(2026\)](#):

$$\text{div}_i(\alpha) = \sum_{j:(i,j) \in \mathcal{A}} \alpha_{ij} - \sum_{j:(j,i) \in \mathcal{A}} \alpha_{ji}.$$

The dual of (1.2) is

$$\begin{aligned} \min \quad & c\lambda + \sum_i \mu_i \\ \text{s.t.} \quad & w_i\lambda + \mu_i + \text{div}_i(\alpha) \geq p_i \quad i \in \mathcal{V}, \\ & \lambda \geq 0, \quad \mu_i \geq 0, \quad \alpha_{ij} \geq 0. \end{aligned} \tag{1.4}$$

The multiplier λ prices capacity; μ_i prices the upper bound. Later, s_{ij} will denote a *primal slack*, not an arc multiplier. Keeping s_{ij} , α_{ij} and λ distinct prevents a collision with the notation in the historical forest notes.

1.6 Where this sits in the literature

Nothing in (1.1) or in the forest structure that follows from it is new, and it is better to say so here than to let a reader discover it later. The companion document in `literature/` does this carefully; the four blocks are:

Closure and its reduction to flow.

Maximum closure and its solution by minimum cut are classical ([Picard, 1976](#); [Picard and Queyranne, 1982](#)); the pseudoflow algorithm ([Hochbaum, 2008](#)) is what a flow-based competitor would use today, and [Hochbaum and Queyranne \(2003\)](#) place the closure problem and isotonic regression in one family.

The ratio objective and the decomposition.

The normalization is [Charnes and Cooper \(1962\)](#); the parametric reading is [Dinkelbach \(1967\)](#); the decomposition into blocks of decreasing ratio is [Sidney \(1975\)](#) and [Lawler \(1978\)](#); parametric flow over the whole ratio range is [Gallo et al. \(1989\)](#) and [Eppstein \(2018\)](#), and the divide-and-conquer scheme is [Eisner and Severance \(1976\)](#). The union convention that makes a block canonical is the maximal source side of a minimum cut ([Picard and Queyranne, 1980](#)).

Forests carrying branch aggregates.

This is the oldest and closest antecedent. The open-pit design algorithm of [Lerchs and Grossmann \(1965\)](#) maintains a *normalized tree*, carries the mass of each branch, classifies an arc by the sign of the mass it supports, and pivots by adding one arc and removing a branch's root arc; [Hochbaum \(2001\)](#) sets this out and recovers a flow from the tree in linear time, and also gives a parametric variant with the cost of a single run. Substituting $b_i = p_i - \rho w_i$ turns the strong/weak test into the sign of the reduced costs derived in Section 4.

Recent simplex work on the same shapes.

[Yang \(2026\)](#) develops the revised-simplex algebra on rooted spanning forests for budget-constrained total-variation problems, which by reading (c) of Section 2.2 is the same structural family as (2.2). [Wuille \(2025\)](#) states (2.2) itself, with $n - 1$ nonbasic variables, a spanning forest of tight dependency slacks, at most one free vertex variable per component and a single positive component; [Mollakhani et al. \(2026\)](#) formalize that as a spanning-forest linearization algorithm. Simplex methods on an incidence matrix with one side row are classical ([Ahuja et al., 1993](#); [Schrijver, 2003](#); [Holzhauser et al., 2017](#)).

What this tutorial develops is therefore a *specialization*: the ordinary primal simplex carried through exactly on (2.2), with the closed forms that specialization produces. It is not a new class of basis and not a new paradigm.

1.7 Cycles and limits of the model

If the input digraph contains a directed cycle, the corresponding inequalities force equality of the variables on that cycle. Contract each strongly connected component, sum its profits and weights, and retain a map to expand the final solution. The resulting graph is a DAG with positive aggregated weights. Thus the DAG assumption is a preprocessing convention for general directed precedence graphs. In contrast, zero or negative weights, several independent capacity rows, heterogeneous upper bounds, and inequalities of the form $x_i \leq \gamma_{ij} x_j$ require additional analysis. They are not included in the statements of this draft.

2 The ratio oracle and its normalization

Suppose a closed prefix M has already been selected. Put $R = \mathcal{V} \setminus M$ and retain only arcs whose two endpoints lie in R . A set $S \subseteq R$ is a closure in this residual graph if and only if $M \cup S$ is a closure in the original graph. Arcs from R to M already have their prerequisites satisfied. The next subproblem is

$$\rho^*(R) = \max \left\{ \frac{p(S)}{w(S)} : \emptyset \neq S \subseteq R, S \text{ a residual closure} \right\}. \quad (2.1)$$

We now work on one nonempty residual graph, suppress the superscript R , and use n, m for its size.

2.1 Normalization removes the quotient

The ratio-closure simplex solves the following LP:

$$\begin{aligned}
 \max \quad & \sum_i p_i y_i \\
 \text{s.t.} \quad & \sum_i w_i y_i = 1, \\
 & y_i - y_j \leq 0 \quad (i, j) \in \mathcal{A}, \\
 & y_i \geq 0 \quad i \in \mathcal{V}.
 \end{aligned} \tag{2.2}$$

These y_i are normalized variables. They have *no upper bound of one*. For a nonempty closure S , the vector $\mathbf{y} = \chi^S/w(S)$ is feasible and has objective $\varrho(S)$. Its coordinates can exceed one if $w(S) < 1$.

Theorem 2.1 (Exact ratio normalization). *The feasible set of (2.2) is nonempty and compact. Its optimal value is (2.1), including when this value is negative.*

Proof. The full vertex set is a nonempty closure. Positivity of the weights implies $0 \leq y_i \leq 1/w_i$, so the feasible set is bounded and closed. For any feasible y , define $C_t = \{i : y_i \geq t\}$ for $t > 0$. Each C_t is a closure, and the finite step function representation gives

$$y = \int_0^\infty \chi^{C_t} dt, \quad 1 = \int_0^\infty w(C_t) dt, \quad \mathbf{p}^T \mathbf{y} = \int_0^\infty p(C_t) dt.$$

For every nonempty C_t , $p(C_t) \leq \rho^* w(C_t)$; for an empty set both sides vanish. Integrating proves $\mathbf{p}^T \mathbf{y} \leq \rho^*$. Conversely, $\chi^S/w(S)$ attains the ratio of any maximizing closure. $\square$

2.2 The class of linear program this is

Three equivalent readings of (2.2) fix its place among linear programs, and each is used later.

- (a) **One budget row over a homogeneous difference system.** The inequalities $y_i - y_j \leq 0$ are homogeneous difference constraints on a digraph; one dense equality row with positive coefficients is added.
- (b) **The base of an order cone.** The set $\{\mathbf{y} \geq 0 : y_i \leq y_j\}$ is the cone of nonnegative order-preserving vectors on the poset, whose extreme rays are the characteristic vectors of the nonempty closures; the equality slices that cone into the polytope with vertices $\chi^C/w(C)$. Optimizing over it therefore returns $\max_C \varrho(C)$, which is Theorem 2.1 read geometrically. It is the cone analogue of Stanley's order polytope, whose vertices are the characteristic vectors of the filters of the poset (Stanley, 1986).
- (c) **Transposed incidence plus one dense row.** Each difference constraint carries a +1 and a -1 in the columns of two vertex variables, so the matrix is the *transpose* of a digraph incidence matrix with one dense row appended, and the right-hand side vanishes on the network part. This is $\mathbf{A}$ in (2.4) below.

Reading (c) is worth keeping in mind. In the side-constrained network flow literature the variables are arcs and the rows are vertices, and here it is the other way round; the two formulations are linear programming duals of one another, so the algebra transports between them, but pricing on one side corresponds to the ratio test on the other. The same duality places (2.2) in the structural family of isotonic regression and of total variation on a graph, which are difference systems on vertex variables as well — which is why Yang (2026) is close prior art.

An inequality $\mathbf{w}^T \mathbf{y} \leq 1$ would allow $y = 0$ and return zero when every nonempty closure has negative profit. The equality in (2.2) is therefore deliberate. The outer capacitated LP may choose nothing, but the ratio oracle must still identify the best *nonempty* closure.

2.3 Standard form and the ratio dual

Introduce $s_{ij} = y_j - y_i \geq 0$. The standard form is

$$\mathbf{w}^T \mathbf{y} = 1, \quad y_i - y_j + s_{ij} = 0 \ ((i, j) \in \mathcal{A}), \quad (y, s) \geq 0. \quad (2.3)$$

There are $n + m$ variables and $m + 1$ independent equations: the slack columns contain an identity matrix, and the normalization row is independent of the arc rows. A basis has $m + 1$ variables and its complement has $n - 1$. With $\mathbf{D}$ denoting the arc-by-vertex matrix with row $\mathbf{e}_i^T - \mathbf{e}_j^T$, write

$$\mathbf{A} = \begin{pmatrix} \mathbf{w}^T & 0 \\ \mathbf{D} & \mathbf{I}_m \end{pmatrix}, \quad \mathbf{b} = (1, 0, \dots, 0)^T, \quad \mathbf{g} = (\mathbf{p}, 0). \quad (2.4)$$

This matrix is used for independent checking, not stored densely by the proposed efficient implementation.

The dual is

$$\begin{aligned} \min \quad & \beta \\ \text{s.t.} \quad & w_i \beta + \text{div}_i(\alpha) \geq p_i \quad i \in \mathcal{V}, \\ & \alpha_{ij} \geq 0 \quad (i, j) \in \mathcal{A}, \quad \beta \in \mathbb{R}. \end{aligned} \quad (2.5)$$

Here β is free because normalization is an equality. For any closure S , summing the inequalities gives

$$p(S) \leq \beta w(S) + \sum_{i \in S} \text{div}_i(\alpha) \leq \beta w(S).$$

The last inequality holds because no arc leaves a closure and nonnegative flow may enter it. Thus a feasible dual vector provides a direct certificate of an upper bound on every closure ratio.

3 A basis represented by a forest

The central combinatorial object is a pair $(\mathcal{F}, \mathcal{Q})$: an undirected forest formed from selected precedence arcs, retaining each arc's original orientation, and a set of root vertices. Every tree has one root except for a single rootless tree. The theorem below establishes that this object encodes exactly the nonsingular bases of the normalized LP. Its feasible instances are then identified by a closure condition.

Remark 3.1 (The forest is not new). A forest carrying per-branch aggregates, with branches classified by the sign of the aggregate they support, is the state maintained by the open-pit design algorithm of [Lerchs and Grossmann \(1965\)](#): its *normalized tree* hangs branches from an added root, records the mass $b(T_q)$ of each branch, calls a downward arc *strong* when the mass it supports is positive, and pivots by adding one arc and removing a branch's root arc. [Hochbaum \(2001\)](#) sets that algorithm out in this language and shows the tree determines a flow in linear time. The same forest structure appears for the normalized ratio program in [Wuille \(2025\)](#), and the revised-simplex algebra on rooted spanning forests is developed in [Yang \(2026\)](#) for a different linear program. What follows is therefore a specialisation of the ordinary primal simplex to (2.3), not a new class of basis; the positioning document in `literature/` states precisely what that leaves and what it does not.

3.1 Nonbasic slacks are the forest edges

Let $\mathcal{N}$ denote the nonbasic variables in (2.3), and define

$$\mathcal{F} = \{(i, j) : s_{ij} \in \mathcal{N}\}, \quad \mathcal{Q} = \{i : y_i \in \mathcal{N}\}.$$

The basic variables are recovered from the object by taking the complement:

$$\mathcal{B} = \{y_i : i \notin \mathcal{Q}\} \cup \{s_e : e \notin \mathcal{F}\}.$$

Thus a forest edge represents a *nonbasic slack*, and a root represents a *nonbasic vertex variable*. Every remaining variable is basic, including those whose value happens to be zero. The representation records which tight constraints define the basis, even when additional constraints are also tight.

Setting the nonbasic variables to zero gives

$$y_i = y_j \quad ((i, j) \in \mathcal{F}), \quad y_q = 0 \quad (q \in \mathcal{Q}). \quad (3.1)$$

Consequently, the vertex values are constant on each connected component of $(\mathcal{V}, \mathcal{F})$, with directions ignored only for connectivity. The original orientation remains necessary when slack variables move.

Theorem 3.2 (Forest characterization of bases). *A set of $n - 1$ nonbasic variables is the complement of a nonsingular basis of (2.3) if and only if:*

- (i) $\mathcal{F}$ is a spanning forest, with isolated vertices allowed;
- (ii) each of its trees has at most one vertex in $\mathcal{Q}$;
- (iii) exactly one tree T_* has no such vertex; every other tree has one.

A vertex in $\mathcal{Q}$ is called the root of its tree.

Proof. Eliminating all slacks leaves the normalization row and the $n - 1$ equations in (3.1), a square system on y . A cycle among forest edges would make its edge-difference rows dependent. Likewise, two roots in one component would be dependent modulo the path equations. Hence nonsingularity requires an acyclic edge set and at most one root per component. If the forest has k trees, it has $n - k$ edges. Counting $n - 1$ nonbasic variables then requires $k - 1$ roots, leaving exactly one tree rootless. Conversely, the edge equations reduce y to one scalar per tree. The $k - 1$ roots fix all but one scalar to zero, and normalization fixes the remaining scalar uniquely because $w(T_*) > 0$. $\square$

At the resulting basic solution,

$$y_i = \begin{cases} 1/w(T_*), & i \in T_*, \\ 0, & i \notin T_*, \end{cases} \quad s_{ij} = y_j - y_i, \quad \rho = \mathbf{p}^T \mathbf{y} = \frac{p(T_*)}{w(T_*)}. \quad (3.2)$$

We call T_* the positive tree and all other trees zero trees.

Corollary 3.3 (Feasibility and support). *The forest basis is primal feasible if and only if T_* is a closure. Every feasible basis has exactly one positive vertex level, and its support is the connected closure T_* .*

Proof. The only way a slack in (3.2) can be negative is an arc whose tail is in T_* and whose head lies outside it. Excluding those arcs is exactly the closure condition. $\square$

This characterization concerns the normalized oracle, not every optimum of the original capacitated LP. Moreover, a *basic* variable can be zero. An arc outside $\mathcal{F}$ has a basic slack, but that slack is zero whenever its endpoints have the same value. These additional tight arcs account for many degenerate pivots.

3.2 A simple feasible initialization

A nonempty DAG has a sink v , meaning that no arc leaves v under our prerequisite convention. Choose

$$\mathcal{F} = \emptyset, \quad \mathcal{Q} = \mathcal{V} \setminus \{v\}, \quad T_* = \{v\}. \quad (3.3)$$

The basic variables are y_v and all arc slacks. Set $y_v = 1/w_v$ and all other vertex variables to zero. The singleton $\{v\}$ is a closure, so this basis is feasible. This construction works on disconnected graphs and needs no artificial arcs or phase-I problem. For the numerical trace, choose the smallest-index sink, vertex 1.

3.3 Rooting and the orientation sign

Root each zero tree at its root; root T_* at any vertex r_* . For a tree edge e , let a be its parent endpoint and b its child endpoint. Let D_e be the subtree rooted at b , including b , and put

$$\sigma_e = \begin{cases} +1, & e = (a, b) \text{ points from parent to child,} \\ -1, & e = (b, a) \text{ points from child to parent.} \end{cases} \quad (3.4)$$

The slack equation implies

$$y_b - y_a = \sigma_e s_e. \quad (3.5)$$

Thus a vertex value is its root value plus the signed sum of slacks on its root-to-node path. On a general graph these signs depend on orientations, not on path-length parity.

4 Reduced costs and directions from subtree sums

We now express one revised simplex iteration using the forest. The positive tree is T_* , its weight is $W_* = w(T_*)$, and the current objective is $\rho = p(T_*)/W_*$. All formulas refer to the current basis.

4.1 Connection with revised simplex: the two basis systems

Write $\mathbf{B} = \mathbf{A}_{\mathcal{B}}$ and let $\mathbf{g} = (\mathbf{p}, 0)$ be the standard-form objective vector. The two familiar basis systems in a primal revised simplex iteration are

$$\mathbf{B}^T \boldsymbol{\pi} = \mathbf{g}_{\mathcal{B}}, \quad \bar{c}_q = g_q - \mathbf{A}_q^T \boldsymbol{\pi}, \quad \text{multipliers and reduced costs,} \quad (4.1)$$

$$Bv = A_q, \quad d_{\mathcal{B}} = -v, \quad \text{column for entering variable } q. \quad (4.2)$$

The forest supplies explicit solutions to both systems. For the first, subtree sums yield the reduced costs in (4.6)–(4.7) and the multipliers $\boldsymbol{\pi} = (\rho, \alpha)$ in (4.11). For the second, a signed tree or subtree movement and one normalization correction give (4.9). Thus neither stage requires a general basis factorization or a triangular solve. The next subsections derive these identities and explain their arithmetic costs. In this precise sense, Forest Simplex implements the revised simplex operations through a combinatorial representation of the basis: it computes the same multipliers, reduced costs, and pivot directions by forest traversals and aggregate formulas.

The entering variable selects a *column*; the ratio test subsequently selects the leaving variable and hence a *row*. If that pivot row itself is required, its usual transposed basis system also has a direct forest evaluation, given in (4.10) below.

4.2 The dictionary behind the formulas

Let a_T be the root variable of tree T . In zero trees this is the nonbasic root y_{q_T} ; in the positive tree it is basic. Repeated use of (3.5) gives

$$y_i = a_{T(i)} + \sum_{e \text{ on root-to-} i \text{ path}} \sigma_e s_e. \quad (4.3)$$

Summing with weights, each s_e appears exactly at the descendants D_e . Therefore

$$a_{T_*} = \frac{1 - \sum_{T \neq T_*} w(T) y_{q_T} - \sum_{e \in \mathcal{F}} \sigma_e w(D_e) s_e}{W_*}. \quad (4.4)$$

Equations (4.3)–(4.4), followed by $s_{ij} = y_j - y_i$ for nonforest arcs, are an explicit simplex dictionary. They express every basic variable in terms of the $n - 1$ nonbasic variables. Only the positive root is eliminated by normalization.

Substituting into the objective yields

$$\begin{aligned} \mathbf{p}^T \mathbf{y} = \rho + \sum_{T \neq T_*} [p(T) - \rho w(T)] y_{q_T} \\ + \sum_{e \in \mathcal{F}} \sigma_e [p(D_e) - \rho w(D_e)] s_e. \end{aligned} \quad (4.5)$$

This identity immediately gives all reduced costs, without solving a dense linear system.

Proposition 4.1 (Forest pricing). *For the maximization problem, the reduced costs of the nonbasic variables are*

$$\bar{c}_{y_{q_T}} = p(T) - \rho w(T), \quad T \neq T_*, \quad (4.6)$$

$$\bar{c}_{s_e} = \sigma_e [p(D_e) - \rho w(D_e)], \quad e \in \mathcal{F}. \quad (4.7)$$

A positive reduced cost identifies a candidate entering variable. It does not guarantee a positive feasible step.

Specialized entering-variable selection. A postorder traversal computes all subtree profit and weight sums in $O(n)$ arithmetic operations. With these aggregates available, either formula above costs $O(1)$ per candidate. There are exactly $n - 1$ nonbasic variables, so full pricing and Bland's entering-variable selection cost $O(n)$, independently of the number m of original arcs. No arc outside the forest needs pricing. Writing $P_* = p(T_*)$, signs and comparisons of reduced costs can also use their numerators over the common positive denominator W_* :

$$W_* p(T) - P_* w(T), \quad \sigma_e [W_* p(D_e) - P_* w(D_e)].$$

These comparisons avoid division; they do not remove the need to control overflow or rounding in a numerical implementation.

4.3 A specialized pivot column from one component movement

Choose a nonbasic variable q and increase it from zero to t . Define the unadjusted vertex displacement h by

$$h = \begin{cases} \chi^T, & q = y_{q_T} \text{ is a root,} \\ \sigma_e \chi^{D_e}, & q = s_e \text{ is a forest slack.} \end{cases} \quad (4.8)$$

The raw motion generally changes total weight. Restore normalization by shifting every vertex of T_* by the same amount:

$$\kappa = \frac{\sum_i w_i h_i}{W_*}, \quad d_i = h_i - \kappa \chi_i^{T_*}, \quad d_{s_{ij}} = d_j - d_i \quad ((i, j) \in \mathcal{A}). \quad (4.9)$$

The full candidate motion is $y_i(t) = y_i + t d_i$ and $s_{ij}(t) = s_{ij} + t d_{s_{ij}}$.

Proposition 4.2 (The forest motion is the simplex direction). *The direction in (4.9) has component one at the entering variable, zero at every other nonbasic variable, and satisfies all homogeneous equations of (2.3). Consequently, its basic components equal $-A_B^{-1}A_q$, and its objective derivative is $\bar{c}_q$.*

Proof. For a root entry, h is constant on each tree, and the chosen root increases by one. All other roots remain fixed. For an edge entry, h is constant on each side of the deleted tree edge and its head-minus-tail difference on that edge is one by (3.4). No other forest edge has a nonzero head-minus-tail difference. No zero-tree root belongs to D_e . The correction on T_* is constant on that entire tree and affects neither these differences nor any zero-tree root. By construction $\mathbf{w}^T \mathbf{d} = 0$, and the slack differences enforce every arc equation. Uniqueness follows from nonsingularity of the basis. Finally,

$$\mathbf{p}^T \mathbf{d} = \mathbf{p}^T \mathbf{h} - \rho \mathbf{w}^T \mathbf{h},$$

which gives (4.6) or (4.7). $\square$

Computing the column without a basis solve. The conventional tableau column is $A_B^{-1}A_q = -d_B$. Equation (4.9) computes it directly from the forest, including its sign convention. For root entry, $\kappa = w(T)/W_*$; for slack entry, $\kappa = \sigma_e w(D_e)/W_*$. Each uses a stored aggregate. The direction is described by the selected tree or subtree, its sign, the positive tree, and one scalar. Once component membership is available and the selected subtree is marked, each vertex coordinate and each slack coordinate costs $O(1)$ to evaluate. Marking and forming all vertex coordinates cost $O(n)$; explicitly forming all slack coordinates costs $O(m)$. The ratio test can evaluate these coordinates as it scans the basic variables, so storing a separate complete pivot column is optional.

The pivot row, explicitly. Suppose the ratio test chooses basic variable z_b , at position r in the ordered basis. A conventional way to obtain its tableau row is to solve $\mathbf{B}^T \boldsymbol{\eta} = \mathbf{e}_r$ and form $a_{rq} = \boldsymbol{\eta}^T \mathbf{A}_q$. For each nonbasic candidate q , let $h^{(q)}$ and κ_q be the quantities in (4.8)–(4.9). The same forest formulas give directly

$$a_{rq} = -d_b^{(q)} = \begin{cases} \kappa_q \chi_i^{T*} - h_i^{(q)}, & z_b = y_i, \\ h_i^{(q)} - h_j^{(q)} + \kappa_q (\chi_j^{T*} - \chi_i^{T*}), & z_b = s_{ij}. \end{cases} \quad (4.10)$$

Indeed, $d_B^{(q)} = -B^{-1}A_q$, so its component at position r is the negative of the requested row coefficient. Component identifiers and depth-first entry/exit indices allow tree and subtree membership queries in $O(1)$ after $O(n)$ forest preprocessing. With the aggregates also available, each coefficient above costs $O(1)$ and all $n - 1$ nonbasic coefficients of a chosen row cost $O(n)$. The basic coefficients are the corresponding identity row. The reference primal algorithm only needs the entering column and ratio test; (4.10) makes explicit how to recover the row when needed.

Why all arcs still matter. Only forest arcs are needed for pricing and vertex directions. However, the basic slack on *every* nonforest arc can decrease. Those arcs must be considered in the ratio test even when they join two different trees or are transitively redundant. Ignoring them can produce an infeasible pivot.

4.4 Reading a dual certificate from the forest

Define arc numbers and vertex residuals by

$$\beta = \rho, \quad \alpha_e = \begin{cases} -\bar{c}_{s_e}, & e \in \mathcal{F}, \\ 0, & e \notin \mathcal{F}, \end{cases} \quad (4.11)$$

$$r_i = w_i \rho + \operatorname{div}_i(\alpha) - p_i. \quad (4.12)$$

Proposition 4.3 (Optimality certificate). *For a forest basis, $r_i = 0$ at every basic vertex variable and $r_{q_T} = -\bar{c}_{y_{q_T}}$ at every root. Thus if all nonbasic reduced costs are nonpositive, (4.11) is feasible for (2.5) and certifies the optimal ratio ρ .*

Proof. Set $u_i = p_i - \rho w_i$. On any rooted subtree D_e , the only forest edge crossing its boundary is e . Its net outgoing flow is $-\sigma_e \alpha_e = u(D_e)$ by (4.7). Taking differences between a subtree and its child subtrees proves $\operatorname{div}_i(\alpha) = u_i$ at every nonroot vertex. On T_* , $u(T_*) = 0$, so the same equality holds at its root. In a zero tree, all nonroot equalities hold and the residual at the root is $-u(T) = \rho w(T) - p(T)$. Nonpositive reduced costs make every α_e and every r_i nonnegative. The primal and dual objectives both equal ρ , proving optimality. $\square$

This certificate uses sums on the original arcs of the residual instance. It is suitable for a verifier independent of the data structures used to update the forest. At every forest basis, even before optimality, the vanishing residuals at basic vertex variables and the zero multipliers on basic slacks imply $\mathbf{B}^T \boldsymbol{\pi} = \mathbf{g}_B$. Thus the construction also solves the first basis system (4.1); nonpositive reduced costs are needed only to make these multipliers dual feasible.

5 The ratio test, forest updates, and termination

5.1 The largest admissible step

For brevity let $\mathbf{z} = (\mathbf{y}, \mathbf{s})$ denote the standard-form variable vector in this section only, and let d include vertex and slack components from (4.9). With entering variable q , every other nonbasic variable stays zero. Nonnegativity of the basic variables gives

$$t^* = \min_{b \in \mathcal{B}: d_b < 0} \frac{z_b}{-d_b}. \quad (5.1)$$

Choose a leaving variable attaining this minimum, and set

$$\mathbf{z}' = \mathbf{z} + t^* d, \quad \mathcal{N}' = (\mathcal{N} \setminus \{q\}) \cup \{b\}. \quad (5.2)$$

The set in (5.1) cannot be empty: the full normalized polytope, including its slacks, is bounded, whereas $d_q = 1$ would otherwise yield an infinite feasible ray. The new objective is $\rho' = \rho + t^* \bar{c}_q$.

If a blocking basic variable is already zero, $t^* = 0$. Such a pivot changes the basis and the forest but not the point. It can be necessary even when the current point already has the optimal objective value.

5.2 All four structural cases

Forest edges encode *nonbasic* slacks. Hence an entering slack is removed from the forest, whereas a leaving slack is added to it. Roots obey the corresponding rule for vertex variables. Removing a forest edge splits a tree; adding a leaving slack joins the appropriate components after any split. A root exchange can transfer which component is positive. One should not guess these changes from an improvement in the objective: the ratio test determines the valid exchange.

Entering	Leaving	Edge update	Root update
y_q	y_b	$\mathcal{F}' = \mathcal{F}$	$\mathcal{Q}' = (\mathcal{Q} \setminus \{q\}) \cup \{b\}$
y_q	s_f	$\mathcal{F}' = \mathcal{F} \cup \{f\}$	$\mathcal{Q}' = \mathcal{Q} \setminus \{q\}$
s_e	y_b	$\mathcal{F}' = \mathcal{F} \setminus \{e\}$	$\mathcal{Q}' = \mathcal{Q} \cup \{b\}$
s_e	s_f	$\mathcal{F}' = (\mathcal{F} \setminus \{e\}) \cup \{f\}$	$\mathcal{Q}' = \mathcal{Q}$

Table 2: Updates determined by the entering and leaving variable types.

Theorem 5.1 (Correctness of one pivot). *Starting from a feasible forest basis, a positive reduced-cost entry and the full ratio test (5.1) produce a feasible basis represented by Table 2. The objective does not decrease.*

Proof. Proposition 4.2 identifies exactly the ordinary simplex direction. The ratio test preserves nonnegativity of every basic variable; equality constraints and the remaining nonbasic zero values are preserved by the direction. The pivot element is nonzero because $d_b < 0$, so exchanging the columns yields a nonsingular basis. Its nonbasic complement is exactly (5.2), hence Theorem 3.2 proves the forest and root properties without additional assumptions on orientation. The objective change is $t^* \bar{c}_q \geq 0$. $\square$

Rebuilding the new components and evaluating (3.2) must reproduce z' . This is a particularly useful implementation check. A separate heuristic “tree feasibility” test cannot replace the complete ratio test, which includes all decreasing vertex variables and all decreasing basic slacks.

5.3 A finite reference algorithm

Fix a total ordering of all vertex and slack variables. For the reference version use Bland’s rule: the smallest-index nonbasic variable with $\bar{c}_q > 0$ enters; among all minimum-ratio blockers the smallest-index basic variable leaves. Both parts of this rule matter for the standard finiteness guarantee (Bland, 1977).

Algorithm 1 Ratio-closure simplex ratio oracle, exact-arithmetic reference

Require: Nonempty DAG, profits p , positive weights w , fixed variable order

Ensure: Optimal ratio ρ , an optimal connected closure T_* , dual certificate

- 1: Choose a sink v ; set $\mathcal{F} \leftarrow \emptyset$, $\mathcal{Q} \leftarrow \mathcal{V} \setminus \{v\}$
 - 2: **loop**
 - 3: Recover trees, roots and T_* ; root the trees
 - 4: Compute subtree p, w sums, basic values, and ρ from (3.2)
 - 5: Price all nonbasic variables using (4.6)–(4.7)
 - 6: **if** all reduced costs are nonpositive **then**
 - 7: **return** ρ, T_* and the certificate (4.11)
 - 8: **end if**
 - 9: Select the smallest-index improving nonbasic variable q
 - 10: Compute h , vertex directions and all slack directions by (4.9)
 - 11: Compute t^* and the smallest-index tied leaving variable b by (5.1)
 - 12: Update $\mathcal{F}, \mathcal{Q}$ using Table 2
 - 13: **end loop**
-

Theorem 5.2 (Finite exact-arithmetic oracle). *Algorithm 1 terminates at an optimum of (2.1).*

Proof. Initialization is feasible by (3.3). Each iteration is the ordinary primal simplex algorithm with Bland’s entering and leaving rules, by Proposition 4.2 and Theorem 5.1. The normalized polytope is nonempty and bounded. Bland’s finite pivoting theorem therefore applies. At termination Proposition 4.3 provides an optimality certificate, and Theorem 2.1 identifies its value with the closure ratio. $\square$

This is a finite-pivot theorem in exact arithmetic, not a polynomial bound on the number of pivots, and not a guarantee for a floating-point variant with a different pricing rule. Such variants require their own safeguards.

6 A complete execution on the running instance

6.1 Variable order and the eight pivots

Order $y_1, \dots, y_8$ first, followed by the nine slacks in the arc order (1.3). Initialize with sink 1: $y_1 = 1$, all other vertex variables zero, $\mathcal{F} = \emptyset$, and roots $2, \dots, 8$. The initial ratio is 1. The exact reference execution is shown in Table 3.

Pivot	Enter	Leave	$\bar{c}_q$	t^*	ρ before	ρ after
1	y_5	$s_{5,2}$	4	0	1	1
2	y_2	$s_{5,6}$	2	0	1	1
3	y_6	$s_{6,3}$	6	0	1	1
4	y_3	$s_{5,1}$	4	$\frac{1}{6}$	1	$\frac{5}{3}$
5	y_7	$s_{7,4}$	$\frac{1}{3}$	0	$\frac{5}{3}$	$\frac{5}{3}$
6	y_8	$s_{8,7}$	$\frac{4}{3}$	0	$\frac{5}{3}$	$\frac{5}{3}$
7	$s_{5,6}$	y_1	$\frac{2}{3}$	$\frac{1}{2}$	$\frac{5}{3}$	2
8	$s_{5,1}$	y_2	1	0	2	2

Table 3: Complete Bland-rule trace for the first ratio oracle. Each row is checked against a separately assembled rational basis system.

Only pivots 4 and 7 have a positive step. The first three pivots make dependencies tight before vertices 5 and 6 can increase. Pivots 5 and 6 also reorganize a zero region. Pivot 8 is needed to remove a positive reduced cost from a degenerate optimal basis.

6.2 A first zero step: a prerequisite blocks motion

At the initial basis, y_5 has reduced cost $6 - 1 \cdot 2 = 4$. Its raw displacement is $h = \chi^{\{5\}}$, and normalization gives $d_5 = 1$, $d_1 = -2$, with all other vertex directions zero. Consequently

$$d_{s_{5,1}} = -3, \quad d_{s_{5,2}} = -1, \quad d_{s_{5,6}} = -1.$$

Although $s_{5,1} = 1$, both $s_{5,2}$ and $s_{5,6}$ are zero. The step is zero, and the fixed variable order makes $s_{5,2}$ leave. The new forest contains $(5, 2)$, its zero tree $\{2, 5\}$ is rooted at 2, and the primal point is unchanged. A positive reduced cost alone was insufficient to add vertex 5: its prerequisites had to be handled first.

6.3 The principal split: pivot 7

Immediately before pivot 7, the positive tree and zero tree are

$$T_* = \{1, 2, 3, 5, 6\}, \quad T_0 = \{4, 7, 8\},$$

with $w(T_*) = 6$, $p(T_*) = 10$, and $\rho = 5/3$. Their forest edges are

$$\mathcal{F} = \{(5, 1), (5, 2), (5, 6), (6, 3), (7, 4), (8, 7)\}, \quad \mathcal{Q} = \{4\}.$$

Root T_* at 1. For $e = (5, 6)$, parent 5 points to child 6, so $\sigma_e = +1$ and $D_e = \{3, 6\}$. Its aggregates are $p(D_e) = 4$, $w(D_e) = 2$. Therefore

$$\bar{c}_{s_{5,6}} = 4 - \frac{5}{3} \cdot 2 = \frac{2}{3} > 0, \quad \kappa = \frac{2}{6} = \frac{1}{3}.$$

The vertex direction, in vertex order, is

$$d_y = \left(-\frac{1}{3}, -\frac{1}{3}, \frac{2}{3}, 0, -\frac{1}{3}, \frac{2}{3}, 0, 0\right). \quad (6.1)$$

Each positive vertex initially has value $1/6$. The decreasing basic vertex variables y_1, y_2, y_5 all give ratio $1/2$. So does the basic slack $s_{8,5} = 1/6$, whose direction is $d_5 - d_8 = -1/3$. The smallest-index tied blocker is y_1 , hence $t^* = 1/2$. The update removes $(5, 6)$ from $\mathcal{F}$ and adds root 1.

The new positive tree is $\{3, 6\}$, with value $1/2$ at both vertices. The vertices $\{1, 2, 5\}$ form a zero tree rooted at 1. The ratio increases to

$$\frac{5}{3} + \frac{1}{2} \cdot \frac{2}{3} = 2.$$

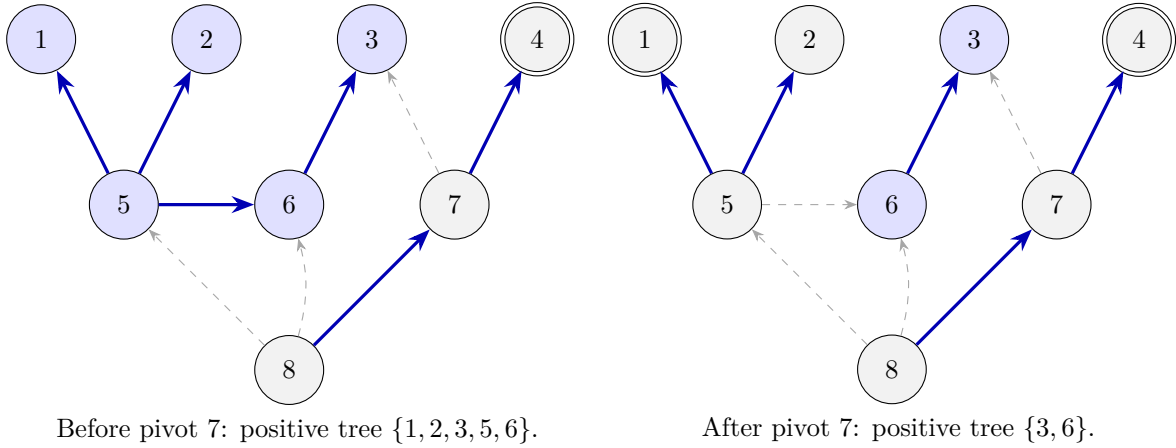


Figure 2: The nonbasic-slack forest in solid blue, other arcs dashed. Positive-tree vertices are shaded blue; double borders mark roots. The right-hand basis still requires the degenerate pivot 8.

6.4 An optimal point need not yet have an optimal basis

After pivot 7, consider edge $(5, 1)$ in the zero tree $\{1, 2, 5\}$, rooted at 1. It points from child 5 to parent 1, so $\sigma = -1$ and $D = \{2, 5\}$. Pricing gives

$$\bar{c}_{s_{5,1}} = -[5 - 2 \cdot 3] = 1 > 0.$$

Its raw motion decreases y_2 and y_5 , which are already zero. The ratio test returns zero and y_2 leaves. The objective remains 2, but the forest now has roots 1, 2, 4 and edges

$$\mathcal{F} = \{(5, 2), (6, 3), (7, 4), (8, 7)\}.$$

All nonbasic reduced costs are now nonpositive. The ratio certificate is

$$\beta = 2, \quad \alpha_{5,2} = 2, \quad \alpha_{6,3} = 3, \quad \alpha_{7,4} = 1, \quad \alpha_{8,7} = 1, \quad (6.2)$$

with every other arc multiplier zero. Its vertex residuals are 1, 1 and 4 at roots 1, 2 and 4, respectively, and zero at all other vertices. It proves that no nonempty closure has ratio greater than 2.

This final pivot is a useful review case: stopping solely because the objective did not change, or because the support already looks correct, would bypass the basis optimality test.

7 From the forest oracle to the capacitated LP

The ratio-closure simplex returns an optimal *connected support* of the normalized oracle. We now explain how repeated calls solve the original LP, how equal ratios are handled, and why this outer construction is valid on general precedence graphs.

7.1 Residual extraction and decreasing ratios

Start with $M = \emptyset$. Call the oracle on the residual graph, obtain its positive tree B , append B to the sequence and replace M by $M \cup B$. The set B is closed on the residual graph, so every accumulated prefix remains closed on the original graph. Each call removes at least one vertex. Thus at most n calls partition the vertex set, even if some ratios are zero or negative. Denote the raw blocks by $B_1, \dots, B_\ell$ and their ratios by $\rho_1, \dots, \rho_\ell$.

Lemma 7.1 (Nonincreasing raw ratios). *The residual extraction procedure satisfies $\rho_{r+1} \leq \rho_r$.*

Proof. The union $B_r \cup B_{r+1}$ is closed in the residual graph before call r . Its ratio is the weighted average of ρ_r and ρ_{r+1} , with strictly positive weights. A value $\rho_{r+1} > \rho_r$ would make this union better than the closure returned at call r , contradicting its certified optimality. $\square$

7.2 Why a single basis may miss a canonical block

For two independent vertices with $p_1 = w_1 = p_2 = w_2 = 1$, either singleton is a valid optimal positive tree. However, the maximum-weight optimal closure is $\{1, 2\}$. A single basic support need not equal a canonical block. Merge consecutive raw blocks with the same ratio. Write the resulting blocks as $\mathcal{K}_1, \dots, \mathcal{K}_k$, their closed prefixes as $\mathcal{M}_r$, and their strictly decreasing ratios as λ_r .

The following proof establishes both the LP connection and the maximal tie convention directly. It is consistent with the classical parametric closure interpretation reviewed in [Dose et al. \(2026\)](#) and [Gallo et al. \(1989\)](#).

Lemma 7.2 (Blockwise closure bound). *For any original closure C and every raw block B_r ,*

$$p(C \cap B_r) \leq \rho_r w(C \cap B_r).$$

Consequently, for every real λ ,

$$u(\lambda) := \max_{C \in \mathcal{C}} \{p(C) - \lambda w(C)\} = \sum_{r=1}^{\ell} w(B_r) \max\{\rho_r - \lambda, 0\}. \quad (7.1)$$

Proof. Both C restricted to the residual graph and B_r are residual closures, and their intersection is one too. If it is nonempty, the oracle certificate at call r bounds its ratio by ρ_r ; otherwise both sums vanish. Hence

$$p(C) - \lambda w(C) \leq \sum_r (\rho_r - \lambda) w(C \cap B_r) \leq \sum_r w(B_r) \max\{\rho_r - \lambda, 0\}.$$

The union of all raw blocks with $\rho_r > \lambda$ is a prefix by Lemma 7.1; it is a closure and attains this bound. $\square$

Corollary 7.3 (Canonical merging). *Merging all consecutive equal-ratio raw blocks produces exactly the maximum-weight tie convention of the supplied manuscript.*

Proof. At any λ , equality in the second inequality of the preceding proof forces an optimal closure to contain every vertex of a block with $\rho_r > \lambda$ and no vertex of a block with $\rho_r < \lambda$; positivity of each vertex weight is used here. The prefix including every block with $\rho_r = \lambda$ is feasible, attains the same value and contains every optimal closure. It is the unique maximum-weight one. At each distinct ratio, its increment is precisely the group merged by the procedure. $\square$

This proof also explains why canonical merging is not an optional heuristic. Equal-ratio blocks may be disconnected. A solver asked for a complete canonical block must finish collecting its ties, even if a requested capacity has already been reached.

7.3 The canonical LP solution

Let q be the last index for which $\lambda_q > 0$, with $q = 0$ when no such index exists. If $w(\mathcal{M}_{h-1}) \leq c < w(\mathcal{M}_h)$ for some $h \leq q$, set

$$\theta = \frac{c - w(\mathcal{M}_{h-1})}{w(\mathcal{K}_h)}, \quad \mathbf{x}^c = \chi^{\mathcal{M}_{h-1}} + \theta \chi^{\mathcal{K}_h}. \quad (7.2)$$

If $c \geq w(\mathcal{M}_q)$, set $\mathbf{x}^c = \chi^{\mathcal{M}_q}$, including the empty solution when $q = 0$.

Theorem 7.4 (Forest-based solution of the original LP). *The vector $\mathbf{x}^c$ above is optimal for (1.2). Within a positive block interval,*

$$z(c) = p(\mathcal{M}_{h-1}) + \lambda_h(c - w(\mathcal{M}_{h-1})). \quad (7.3)$$

For $c \geq w(\mathcal{M}_q)$, $z(c) = p(\mathcal{M}_q)$.

Proof. The vector in (7.2) is a convex combination of two closure indicators, so it respects precedences and box constraints; it uses capacity c . For any feasible $\mathbf{x}$, its threshold sets $C_t = \{i : x_i \geq t\}$, $0 < t \leq 1$, are closures. Thus for any $\lambda \geq 0$,

$$\mathbf{p}^T \mathbf{x} - \lambda \mathbf{w}^T \mathbf{x} = \int_0^1 [p(C_t) - \lambda w(C_t)] dt \leq u(\lambda), \quad \mathbf{p}^T \mathbf{x} \leq u(\lambda) + \lambda c.$$

Choose $\lambda = \lambda_h$ and use (7.1). Both adjacent prefixes attain $u(\lambda_h)$, so $\mathbf{x}^c$ attains the bound. For abundant capacity choose $\lambda = 0$; the positive prefix attains $u(0)$. $\square$

The theorem guarantees a canonical optimum with at most one fractional level; it does not claim that every optimal point has that form. A zero-ratio block may be included without changing the objective if capacity allows. Negative-ratio blocks must not be added merely to fill unused capacity.

Algorithm 2 Block extraction using only ratio-closure simplex oracles**Require:** General precedence instance after SCC contraction**Ensure:** Complete canonical sequence, or its positive part as requested

```

1:  $R \leftarrow \mathcal{V}$ ; sequence  $\leftarrow$  empty
2: while  $R \neq \emptyset$  do
3:    $(\rho, B, \text{certificate}) \leftarrow \text{ratio-closuresimplex}(R)$  using Algorithm 1
4:   if only the positive path is requested and  $\rho \leq 0$  then
5:     break
6:   end if
7:   if last block has ratio  $\rho$  then
8:     Merge B into the last block
9:   else
10:    Append B with ratio  $\rho$ 
11:   end if
12:    $R \leftarrow R \setminus B$ 
13: end while
14: Use (7.2) to answer a prescribed capacity, if requested

```

r	$\mathcal{I}_r$	$p(\mathcal{I}_r)$	$w(\mathcal{I}_r)$	λ_r
1	$\{3, 6\}$	4	2	2
2	$\{1, 2, 5\}$	6	4	$\frac{3}{2}$
3	$\{4, 7, 8\}$	4	4	1

Table 4: Canonical blocks produced by residual ratio-closure simplex calls, checked against exhaustive closure enumeration.

7.4 Finishing the eight-node example

Successive calls and canonical merging give Table 4. At $c = 4$, the first block has weight 2, so half the second block is selected:

$$\theta = \frac{4-2}{4} = \frac{1}{2}, \quad x^c = (\frac{1}{2}, \frac{1}{2}, 1, 0, \frac{1}{2}, 1, 0, 0), \quad z(4) = 4 + \frac{1}{2} 6 = 7.$$

A full dual certificate for (1.4) is

$$\lambda = \frac{3}{2}, \quad \mu_6 = 1, \quad \mu_i = 0 \ (i \neq 6), \\ \alpha_{5,1} = \frac{1}{2}, \quad \alpha_{5,2} = \frac{5}{2}, \quad \alpha_{6,3} = \frac{5}{2}, \quad \alpha_{7,4} = 2, \quad \alpha_{8,7} = \frac{3}{2},$$

with all remaining arc multipliers zero. Its objective is $4 \cdot (3/2) + 1 = 7$. Direct substitution verifies feasibility and complementary slackness. This certificate differs from the normalized certificate (6.2): the original LP has capacity and upper-bound prices, while the ratio oracle has only a free normalization price.

8 Efficient implementation: what the derivation permits

The exact reference algorithm is fully specified above. This section describes the implementable baseline, and then the three refinements that measurement showed to matter. The baseline is not a straw man: it is the configuration the solver still offers as its verifiable reference, and every refinement below is a separate option so that its effect can be measured in isolation.

8.1 A linear-memory reference organization

Store the original directed graph in contiguous endpoint and adjacency arrays, with incoming and outgoing access. Store the forest separately through parent, parent-edge, root and child information. Maintain the nonbasic vertex/slack sets and a mapping between variable IDs and their positions in basic lists. Traverse trees iteratively, so a long chain cannot exhaust the call stack.

A postorder traversal computes $p(D_e)$ and $w(D_e)$ for every forest edge. Component totals provide root reduced costs. Only $n - 1$ nonbasic variables need pricing. After selecting an entry, mark its subtree or tree, form (4.9), and scan all arcs for decreasing basic slacks. Rebuild the forest metadata after the exchange. This deliberately simple version provides a reliable reference before local updates are introduced.

8.2 Specialized operations in one iteration

The forest supplies specialized operations for entering-variable selection, the pivot column, the leaving-variable test, and the basis exchange. Table 5 separates these costs for the full-rebuild reference organization. In particular, the $O(n)$ pricing bound is independent of graph density; the complete direction and ratio test also inspect arcs.

Operation	Forest calculation	Cost
Subtree aggregates	Postorder sums of profits and weights	$O(n)$
One reduced cost	Signed aggregate difference, sums available	$O(1)$
Full pricing and entry	Scan the $n - 1$ nonbasic candidates	$O(n)$
Pivot direction on vertices	Mark a tree/subtree; normalize its motion	$O(n)$
Pivot direction on slacks	Head-minus-tail vertex directions	$O(m)$
Ratio test and exit	Scan decreasing basic vertex/slack variables	$O(n + m)$
Chosen pivot row, if needed	Evaluate its $n - 1$ nonbasic coefficients from forest membership and aggregates	$O(n)$
Basis exchange and rebuild	Exchange edge/root membership; rebuild metadata	$O(n + m)$

Table 5: Arithmetic costs with the current forest available and reusable arrays. Constant-time coordinate evaluation assumes the required sums and membership information have been computed.

Proposition 8.1 (Arithmetic cost of the full-rebuild variant). *Using adjacency lists and reusable arrays, one full rebuild, complete pricing, direction computation, full ratio test and exchange can be performed in $O(n + m)$ arithmetic operations and $O(n + m)$ storage. An oracle performing K pivots costs $O((K + 1)(n + m))$ arithmetic operations.*

Proof. Forest construction and traversals touch $O(n + m)$ stored entries. Subtree sums and pricing touch $O(n)$ entries, since a forest has fewer than n edges. Constructing vertex directions is linear in n , and evaluating all slack directions and blockers is linear in m . Updating membership and rebuilding the adjacency information is linear as well. Arrays store only vertices, arcs, membership, aggregates and temporary vectors. $\square$

This is an arithmetic-operation bound, not a bit-complexity bound for growing rational numbers. Nor does it bound K polynomially. The exact checker deliberately builds dense matrices and therefore does *not* implement the memory bound in Proposition 8.1. For residual calls indexed by r , the analogous total is

$$O\left(\sum_r (K_r + 1)(n_r + m_r)\right),$$

when residual graphs are rebuilt linearly at each call. The number of calls is at most the original n ; no parametric-flow complexity bound is being transferred to the ratio-closure simplex.

8.3 Variants to implement and tune

Parameter family	Candidate variants	Invariant to preserve
Pricing	Full scan; blocked scan; candidate list	A full check before claiming no improving entry
Entering rule	Bland; first or best improving	A documented anti-cycling policy for each alternative
Forest update	Full; local; adaptive rebuild	Same mathematical basis and direction as a full reconstruction
Rebuild trigger	Pivot interval; changed-tree fraction; residual threshold	Recover accuracy without silently changing the problem
Initial basis	Sink singleton; closure of a promising seed; best undirected component	Primal feasibility: the positive component must be closed in the residual
Ratio test	Full scan; restricted to boundary-crossing arcs; stop at the first zero step	Same step, and the same leaving variable whenever Bland’s rule is active
Residual start	Fresh sink basis; validated warm start	Primal feasibility on the new residual graph
Vertex order	Input; topological; seeded	Identical certified objective and canonical decomposition

Table 6: Tuning choices of the implementation. Every row is an explicit parameter, so an ablation can switch off one refinement at a time.

Reducing the number of visited vertices or tested candidates is useful only if it reduces total runtime at the required accuracy. The profiler separates pricing, directions, ratio tests, exchange, rebuilds and canonicalization. The measured ranking of these choices is reported in the experimental report, not here; the one result worth repeating is that the entering rule barely moves the outcome while the treatment of degeneracy moves it by factors.

A caution about termination. First-improving or best-improving pricing is not automatically Bland’s rule. Switching to a finite rule permanently after a documented trigger, or using a separately justified lexicographic implementation, are possible safeguards. An informal fallback that repeatedly switches back to an unproved rule does not inherit the finite-pivot theorem without additional reasoning. The reasoning is supplied in Proposition 8.2 below, because a permanent switch turns out to be expensive.

8.4 Degeneracy is the dominant cost

In a basis the vertex values are constant on the positive component and zero on every other tree. A step is therefore blocked at length zero whenever the blocking variable is one of the many already at zero: a vertex of the entering subtree whose direction component is negative, or a precedence slack between two vertices that are both at zero. Degeneracy is not incidental here; it is what the precedence structure produces.

The effect is large enough to reorganize the implementation around it. On applied instances more than 99% of the pivots of an oracle have zero step; on a family with no arcs at all, where no slack can block, the share is zero. Three consequences follow, and each was measured.

The anti-cycling fallback must not become the operating rule. A permanent switch to Bland’s rule after a run of degenerate pivots engages immediately and never releases, because the run never ends: every entering rule then collapses onto Bland within the first oracle. Releasing the fallback at the first pivot with a positive step, and placing the trigger far away, keeps the improving rule without giving up finiteness.

Proposition 8.2 (Finiteness with a released fallback). *Suppose the solver uses an arbitrary entering rule, switches to Bland’s rule after a fixed number of consecutive zero-step pivots, and returns to the original rule at the first pivot with a positive step. Then the oracle performs finitely many pivots.*

Proof. Call an *episode* a maximal run of iterations during which Bland’s rule is active. Within an episode every pivot has zero step, so the basic solution never changes, and the iterations of the episode are exactly the iterations of Bland’s rule applied to the linear program from the basis at which the episode began; by the finite-pivot theorem for Bland’s rule the episode cannot cycle and must end. An episode ends only with a pivot of positive step, which strictly increases the objective of the normalized subproblem. Since there are finitely many bases, hence finitely many objective values, only finitely many episodes can occur. Between episodes only finitely many pivots are possible for the same reason. $\square$

The ratio test need not scan every arc. By (4.9) the direction is $+\sigma$ on the entering subtree, $-\kappa$ on the positive component and zero elsewhere. An arc whose two endpoints lie outside both sets therefore has zero direction difference and cannot block. It suffices to scan the arcs incident to the entering subtree, together with the arcs that cross the boundary of the positive component; and since the positive component is closed, such an arc has its tail outside it, so scanning the outgoing arcs of the vertices outside the positive component completes the cover.

A zero step is the minimum, so the scan may stop at the first one. No step is negative, so the first blocking variable attaining zero already determines the step. The shortcut must be suspended while Bland’s rule is active, since that rule requires the smallest index among ties. This is the technique the group’s 2023 prototype implemented as an explicit feasibility check during the forest traversal, and it is the single largest saving in the ratio test: on one oracle of a mining instance with 1060 vertices the arcs examined fall from 1 110 883 to 184, at an unchanged number of pivots.

8.5 The dual side

Primal degeneracy and dual degeneracy are different defects, and on this problem they are wildly asymmetric. Primal degeneracy is a per-pivot phenomenon: more than 99% of the pivots of an oracle have zero step. The only source of dual degeneracy is a tie between ratios, which is a per-block phenomenon: on a generated instance of two thousand vertices, ties affect 123 of 267 blocks. Even if every tie produced one degenerate dual pivot, that is a hundred against half a million. The dual is therefore the natural method here, which is also what the classical guidance says: use the dual when the primal is heavily degenerate.

Two structural facts make the dual unusually simple on this problem.

Proposition 8.3 (Primal infeasibility is combinatorial). *In any basis of the normalised subproblem, the vertex variables satisfy $y \geq 0$. Every primal infeasibility is a precedence slack $s_{ij} = y_j - y_i < 0$, that is an arc leaving the positive component P ; and the basis is primal feasible exactly when P is closed.*

Proof. The basic solution takes the value $1/w(P)$ on P and zero on every other tree, so $y \geq 0$ always. For an arc (i, j) , $s_{ij} = y_j - y_i$ is negative only when $y_i > y_j$, which forces $i \in P$ and $j \notin P$. Conversely if P is closed no arc leaves it, all such slacks vanish or are positive, and the basis is feasible. $\square$

Choosing the leaving variable is therefore a search for an unsatisfied prerequisite, with no numerical scan, and the stopping rule is that P is closed. The starting basis is dual feasible by construction: make every vertex its own tree and let P be the vertex of maximum ratio; the reduced cost of any other vertex k is then $w_k(p_k/w_k - \rho) \leq 0$, and there are no forest edges yet. It costs $O(n)$, and it is essentially forced: a singleton start is dual feasible only for the maximum-ratio vertex.

The row of the leaving slack is as local as the primal direction. Entering a nonbasic q by one unit moves every vertex by $\sigma_q \chi[u \in S_q] - \kappa_q \chi[u \in P]$, so the row of the slack of (t, h) is nonzero only for the root of each endpoint's tree, for the forest edges on the two paths to their roots, and through the rank-one term κ_q when exactly one endpoint lies in P . It costs $O(n + \text{depth})$.

What the measurement says. The dual removes the degeneracy it was predicted to remove — the share of zero-step pivots falls from above 99% to between 0% and 21% — but the gain in time is between 1.1 and 2.7, not the two orders of magnitude a naive extrapolation would suggest. The reason is worth recording: the primal performs about half a million pivots of which only some fifteen hundred move the objective, while the dual performs three hundred thousand of which most do. It has traded many null steps for many tiny ones. Choosing the leaving arc by structural criteria rather than by smallest index — most attractive head, largest tree, most negative reduced cost — was tried and does not help, because every violated slack has the same value $-1/w(P)$ and there is nothing for a most-infeasible rule to discriminate.

What remains. With those three changes the ratio test is no longer the bottleneck. Pricing and forest rebuilding are: on a layered DAG with dense connections between levels they account for roughly a third and a half of the solver's time respectively, because both are Θ of the touched part of the graph at every pivot while the pivot itself changes very little. Incremental pricing, updated only on the components a degenerate pivot touches, and updates confined to the modified paths, are the natural next steps; neither is implemented here.

8.6 Numerical checks and output contracts

Floating-point implementations must use declared absolute and relative tolerances, inspect primal and dual residuals in the original units, and reconstruct values when incremental updates drift. Near-equal ratios require particular care: approximate merging must not be mistaken for an exact tie. On integer/rational review cases, exact comparisons provide a useful check.

The intended outputs are: ratio and connected support for an oracle; canonical blocks and breakpoints for a path computation; a feasible x and validated bound/certificate for a capacity solve. Time, iteration or numerical limits must produce distinct states rather than an “optimal” label. Optional internal checks can vary with configuration, but final verification remains mandatory for results used in a scientific comparison.

Once the positive path is known, finding a capacity's split block costs $O(\log k)$ with cumulative weights. Returning only the objective or a compact block description can be correspondingly cheap. Materializing all n vertex coordinates still costs $O(n)$. The many-capacity experiment must include the initial path computation and distinguish these output requirements.

9 What has been checked, and how to review this draft

9.1 Independent exact checks

The companion script `tools/verify/verify_tutorial.py` uses Python’s rational arithmetic and does not depend on a commercial LP solver. It implements the formulas in this draft as a pedagogical checker, not as the planned high-performance solver. At each visited basis it independently assembles A_B and solves

$$A_B z_B = b, \quad A_B d_B = -A_q$$

by rational Gaussian elimination. It compares every nonbasic candidate’s direction and reduced cost with the forest formulas, checks the exchanged basis and verifies the final ratio dual. It also solves the revised simplex multiplier system $\mathbf{A}_B^T \boldsymbol{\pi} = \mathbf{g}_B$ independently at every visited basis and compares its solution with the forest multipliers. At each pivot, it solves $\mathbf{A}_B^T \boldsymbol{\eta} = \mathbf{e}_r$ for the leaving position and checks every nonbasic coefficient of the resulting pivot row against the forest direction.

Separately, exhaustive enumeration lists all closures on the small instance, computes exact maximum ratios and maximal ties, and checks the complete canonical sequence. For the LP value it enumerates capacity intersections of pairs of closure points in the weight–profit plane; this independent oracle does not restrict those pairs to the sequence found by the ratio-closure simplex.

The supplied deterministic check suite contains 37 instances: the eight-node example, four explicit edge cases, and 32 seeded small random DAGs with profits of either sign and positive integer weights. It checks 187 bases and 457 candidate directions. The additional revised simplex comparisons check 187 multiplier systems and 90 pivot rows. The eight-node graph has 36 closures. Its first oracle takes eight pivots, six at zero step, and its capacity-4 LP value is 7. Both numerical certificates displayed in the main text are verified by the checker or its returned forest certificate.

These counts describe a finite review suite, not comprehensive testing of an optimized implementation, not a statistical experiment, and not evidence of a runtime advantage. The seed and hashes of the fixture and verification script are saved with the generated results.

9.2 Rebuilding the tables and PDF

From the project root, run

```
python3 tools/build_tutorial.py
```

The command reruns the exact checks, regenerates numerical tables and forest figures, and compiles this document. The single output PDF is

```
build/tutorial/forest_simplex_tutorial.pdf
```

The generated verification record is

```
build/tutorial/generated/verification.json
```

All intermediate TeX files remain under `build/tutorial`. The source instance, shared notation and bibliography each have one authoritative file. No original project source is altered or required during this rebuild.

9.3 Suggested order for mathematical review

The following checkpoints identify concrete places where an error would affect the proposed general algorithm:

1. Check the prerequisite direction in (1.2c) and the parent-child sign in (3.4).
2. Check the converse as well as the forward implication of the basis characterization, Theorem 3.2.
3. Check that the normalization correction in (4.9) preserves every other nonbasic variable.
4. Check the sign of (4.7), especially when a child points to its parent in a zero tree.
5. Check the inclusion of decreasing nonforest slacks in the ratio test.
6. Follow pivots 7 and 8, including the unchanged objective at pivot 8.
7. Check the sign convention in the forest dual certificate and distinguish it from the capacitated LP dual.
8. Check that merging raw ties yields the maximal, possibly disconnected, canonical block.
9. Check the stop at nonpositive ratios for the capacity-inequality LP.

9.4 Work remaining after this review version

The exact-reference derivation above is accompanied by proofs and finite checks. The following tasks remain open as project deliverables:

- external mathematical review and comparison with existing forest, network-simplex and closure algorithms to determine the actual novelty;
- a C++ implementation with linear-memory data structures;
- verified local updates, alternate pricing and warm starts;
- numerical stress tests beyond the exact small-DAG checker;
- competitive backend comparisons, parameter tuning and reproducible runtime/memory experiments;
- extension beyond the declared positive-weight, single-capacity model.

In particular, this document supplies no invented benchmark results and makes no claim that the eventual optimized solver is faster than parametric flow or a general LP solver.

A Closure and min-cut as comparison tools

This appendix records the alternative closure oracle only to locate the Ratio-closure simplex within the existing framework. Maximum closure admits a minimum-cut representation (Picard, 1976); parametric maximum flow and ratio-closure applications are treated by Gallo et al. (1989). The graph-based predecessor of both is Lerchs and Grossmann (1965), whose normalized tree is a forest with branch aggregates rather than a flow; Hochbaum (2001) recovers a flow from that tree in linear time and gives a parametric variant with the cost of a single run, which makes it a comparison tool on the ratio task as well. These are comparison tools, not subroutines required by Algorithms 1 and 2.

For a fixed real parameter λ , assign vertex value $a_i = p_i - \lambda w_i$. Add source s and sink t , capacities

$$c_{si} = \max\{a_i, 0\}, \quad c_{it} = \max\{-a_i, 0\},$$

and a precedence-arc capacity $H > \sum_i \max\{a_i, 0\}$. Every minimum cut has a source-side vertex set C that is a closure, and its capacity is

$$\sum_i \max\{a_i, 0\} - [p(C) - \lambda w(C)].$$

Thus minimizing the cut maximizes the penalized closure objective. For a fixed λ , this construction works with either sign of the parameter. A linear parametric implementation can use a different normalization on a specified interval, as explained in the supplied manuscript.

At a breakpoint, the closure maximizing the penalized objective need not be unique. In a residual network of a maximum flow, source-reachable vertices give the *minimal* source side. The complement of vertices that can reach t gives the *maximal* source side. A canonical comparison must use the latter convention. For a ratio root, the empty closure can also tie at value zero, so an implementation must preserve or recover a nonempty ratio maximizer.

The planned experiments will compare methods on the same task: one ratio, one capacity, or the full positive path. The worst-case bounds of parametric flow cannot be used as a bound for the number of ratio-closure simplex pivots.

References

- Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993.
- Robert G. Bland. New finite pivoting rules for the simplex method. *Mathematics of Operations Research*, 2(2):103–107, 1977. doi: 10.1287/moor.2.2.103.
- A. Charnes and W. W. Cooper. Programming with linear fractional functionals. *Naval Research Logistics Quarterly*, 9(3–4):181–186, 1962. doi: 10.1002/nav.3800090303.
- Werner Dinkelbach. On nonlinear fractional programming. *Management Science*, 13(7):492–498, 1967. doi: 10.1287/mnsc.13.7.492.
- Valerio Dose, Fabio Furini, and Marco Locatelli. The LP relaxation of the precedence-constrained knapsack problem: Macroitems and a common language for the closure, mining, scheduling, and knapsack literatures. Supplied working manuscript, PAPER_LONG/paper_v1.tex. Source for the problem notation and macroitem conventions, 2026.
- Mark J. Eisner and Dennis G. Severance. Mathematical techniques for efficient record segmentation in large shared databases. *Journal of the ACM*, 23(4):619–635, 1976.
- David Eppstein. The parametric closure problem. *ACM Transactions on Algorithms*, 14(1): Article 2, 2018. doi: 10.1145/3147212. Preliminary version WADS 2015; arXiv:1504.04073.
- Giorgio Gallo, Michael D. Grigoriadis, and Robert E. Tarjan. A fast parametric maximum flow algorithm and applications. *SIAM Journal on Computing*, 18(1):30–55, 1989. doi: 10.1137/0218003.
- Dorit S. Hochbaum. A new–old algorithm for minimum-cut and maximum-flow in closure graphs. *Networks*, 37(4):171–193, 2001. doi: 10.1002/net.1012.
- Dorit S. Hochbaum. The pseudoflow algorithm: A new algorithm for the maximum-flow problem. *Operations Research*, 56(4):992–1009, 2008. doi: 10.1287/opre.1080.0524.
- Dorit S. Hochbaum. Flow is best, fast and scalable: The incremental parametric cut for maximum density and other ratio subgraph problems. In *Proceedings of KDIR*, 2024. URL <https://hochbaum.ieor.berkeley.edu/html/pub/Inc-Para-Cut-max-density-KDIR24.pdf>. Verified on the article’s first page; copy in literature/papers/. Full venue details to be completed, see TO_OBTAIN.md.

- Dorit S. Hochbaum and Maurice Queyranne. Minimizing a convex cost closure set. *SIAM Journal on Discrete Mathematics*, 16(2):192–207, 2003.
- Michael Holzhauser, Sven O. Krumke, and Clemens Thielen. A network simplex method for the budget-constrained minimum cost flow problem. *European Journal of Operational Research*, 259(3):864–872, 2017. Preprint arXiv:1607.02284.
- E. L. Lawler. Sequencing jobs to minimize total weighted completion time subject to precedence constraints. *Annals of Discrete Mathematics*, 2:75–90, 1978. Verified on the article’s first page, copy in literature/papers/.
- Helmut Lerchs and Ingo F. Grossmann. Optimum design of open-pit mines. *Transactions of the Canadian Institute of Mining and Metallurgy*, 68:17–24, 1965. Also in Canadian Mining and Metallurgical Bulletin 58(633), January 1965.
- Arman Mollakhani, Pieter Wuille, and Dongning Guo. Bitcoin mempool linearization, 2026. URL <https://arxiv.org/abs/2607.23787>. v1 26 July 2026, v2 15 August 2026.
- Jean-Claude Picard. Maximal closure of a graph and applications to combinatorial problems. *Management Science*, 22(11):1268–1272, 1976. doi: 10.1287/mnsc.22.11.1268.
- Jean-Claude Picard and Maurice Queyranne. On the structure of all minimum cuts in a network and applications. *Mathematical Programming Studies*, 13:8–16, 1980. doi: 10.1007/BFb0120902.
- Jean-Claude Picard and Maurice Queyranne. Selected applications of minimum cuts in networks. *INFOR: Information Systems and Operational Research*, 20(4):394–422, 1982.
- Project archive. Simplex algorithm for the solution of the macro-object subproblem, n.d. Undated internal notes, PAPER/MARCO/SimplexMacro.tex. Source of the forest representation in the historical project.
- Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, Berlin, 2003.
- Jeffrey B. Sidney. Decomposition algorithms for single-machine sequencing with precedence relations and deferral costs. *Operations Research*, 23(2):283–298, 1975. doi: 10.1287/opre.23.2.283.
- Richard P. Stanley. Two poset polytopes. *Discrete & Computational Geometry*, 1:9–23, 1986.
- Pieter Wuille. Spanning-forest cluster linearization. Delving Bitcoin forum post, February 2025. URL <https://delvingbitcoin.org/t/spanning-forest-cluster-linearization/1419>. Posted 4 February 2025 under the handle sipa.
- Dominic Yang. A specialized simplex algorithm for budget-constrained total variation-regularized problems, 2026. URL <https://arxiv.org/abs/2507.13493>. v1 17 July 2025, v3 18 May 2026.